

Unix-Like File Permission Simulator

Kayden McClung, Ethan Prescott, Christopher Farquer, Lee Carter
East Texas A&M University
Cybersecurity Final Project

Abstract— This report analyzes a C++ Unix-like file permission simulator designed to demonstrate discretionary access control in a simplified file system. The program models users, groups, files, directories, ownership, and ugo permission checks. The final implementation includes an interactive command interpreter, user and group management, a simulated directory tree, file creation and deletion, directory navigation, permission checking for read, write, and execute actions, and permission and ownership modification through `chmod`, `chown`, and `chgrp`. The project shows that Unix-style permissions are compact and effective for basic protection, but also reveals the limits of the ugo model when access must be granted to multiple specific users or groups. Overall, the simulator provides a practical demonstration of operating system security concepts in a safe in-memory environment.

Keywords— *Unix permissions, discretionary access control, filesystem security, C++, users, groups, ACLs*

I. INTRODUCTION

Operating systems protect files and directories by deciding which users can access which resources. In Unix-like systems, the traditional permission model is based on three identity categories: the user that owns the object, the group that owns the object, and all other users. Each category may be granted read, write, and execute permissions. Although this model is compact, it is important in real security decisions because it controls who can view, change, execute, list, alter, or traverse filesystem objects.

The File Permission Simulator is a console application written in C++ that recreates this model in a controlled environment. Instead of changing permissions on the host computer, the program builds a simulated filesystem in memory. A user can create simulated users and groups, create directories and files, switch the active user and group, and test whether actions are allowed based on ownership and permission values. This makes the project safe for experimentation while still connecting directly to operating system security concepts.

The objective of this report is to analyze the final project, explain the security model it implements, and evaluate the correctness and limitations of the design. The report is written for other students in the course, so specialized terms are defined before they are used. The most important ideas are discretionary access control, ugo permissions, sequential permission checking, directory permissions, and Access Control Lists.

II. BACKGROUND: DAC AND UGO PERMISSIONS

The simulator is based on discretionary access control, or DAC. In a DAC system, access decisions depend on users, object ownership, and permission settings associated with a resource. The owner of an object can usually decide who else receives access. This differs from mandatory access control, where a central security policy controls access regardless of the ordinary owner's preference.

Unix-like file systems commonly use the ugo model. The letter `u` refers to the owning user, `g` refers to the owning group,

and `o` refers to all other users. Each category can receive read, write, and execute permissions. These are often written as `r`, `w`, and `x`. They are also represented numerically using `read = 4`, `write = 2`, and `execute = 1`. A permission digit is formed by adding these values. For example, `7` means read, write, and execute; `6` means read and write; `5` means read and execute; and `0` means no permission.

A complete permission value has three digits. The first digit applies to the owner, the second applies to the group, and the third applies to other users. For example, `775` means the owner has full access, the group has full access, and other users have read and execute access but not write access. This same general interpretation is used by Unix tools such as `chmod` [2].

The meaning of permissions depends on whether the object is a file or a directory. For a file, read means the ability to view contents, write means the ability to modify contents, and execute means the ability to run the file. For a directory, read means the ability to list contents, write means the ability to add or remove entries, and execute means the ability to traverse into or through the directory. This distinction is essential because a user may own a file but still be blocked from reaching it if the user lacks execute permission on a parent directory.

III. IMPLEMENTATION OVERVIEW

The project is implemented in C++ using multiple classes that represent the main parts of a Unix-like permission system. The source code includes classes for users, groups, user lists, group lists, filesystem items, files, and directories. The build system uses CMake and compiles the simulator as a C++17 executable [1]. This structure keeps the program understandable because each class has a clear responsibility.

The user class stores a username and user ID. The group class stores a group name and group ID. The `userList` and `groupList` classes manage collections of users and groups with maps keyed by name. These container classes support adding, removing, checking existence, and returning pointers to stored users or groups. This supports commands such as `login`, `joingrp`, `chown`, and `chgrp`, which need to locate existing identities before continuing.

The item class is the base class for both files and directories. It stores the shared security metadata: name, owner username, owner UID, owner group name, owner GID, and permission value. It also implements the core permission-checking logic. This inheritance design is appropriate because files and directories share ownership and permission metadata even though they perform different actions.

The file class inherits from item and adds simulated file content. It supports read, write, and execute behavior. Before reading, writing, or running a file, the file object calls the permission checking logic inherited from item. If the required permission is present, the operation succeeds. Otherwise, the simulator prints an error message and blocks the action.

The directory class also inherits from item, but it stores lists of files and subdirectories. It supports creation, deletion, listing, path display, navigation, search, and cleanup of contained objects. This models a simple directory tree

beginning at a root directory. The simulator does not use the host machine's real filesystem for these operations, which makes the project safer and easier to demonstrate.

IV. COMMAND-LINE BEHAVIOR

The simulator runs through an interactive command-line interface. At startup, the program asks for an initial username and UID. It creates that user, creates a default group with the same name and ID, logs in as that user, and creates the root directory. This startup process gives the simulator an initial user, group, and directory context before ordinary commands are processed.

The final command set includes user and group commands, identity commands, file commands, directory commands, and ownership or permission commands. User and group management includes `adduser`, `deluser`, `listusers`, `addgroup`, `delgroup`, and `listgroups`. Identity commands include `login`, `joingrp`, and `whoami`. File commands include `mkfil`, `rmfil`, `read`, `write`, and `run`. Directory commands include `mkdir`, `rmdir`, `opendir`, `backdir`, `pwd`, and `ls`. Security modification commands include `chmod`, `chown`, and `chgrp` [1].

The command interpreter parses each line of input into tokens, looks up the first token in a command map, and calls the matching handler function. This structure keeps the interface flexible because adding a command mainly requires adding a handler function and mapping the command name to that function. If the command is unknown, the simulator reports an error and tells the user to type help.

The project uses `mkfil` and `rmfil` instead of the real Unix names `touch` and `rm`. This is acceptable because the simulator is not trying to operate on the real host filesystem. Instead, these names make it clear that the command is creating or removing a simulated file inside the program. Likewise, `opendir` and `backdir` implement directory navigation without claiming to be a full replacement for shell path handling.

V. SEQUENTIAL PERMISSION CHECKING

The central technical requirement of the project is to model ugo permission enforcement accurately enough for an educational simulator. The simulator follows the standard sequential check: first compare the current user to the file or directory owner, then compare the current group to the object's owning group, and finally use the other-user permission if neither of the first two categories matches. This order matters because the owner permission digit should be used when the current user owns the object, even if the group digit would give different permissions.

The permission check begins by loading the object's numeric permission value. If the current user's UID matches the stored owner UID, the simulator extracts the owner digit. If not, but the current group's GID matches the stored owner GID, it extracts the group digit. If neither match succeeds, it uses the other digit. Once the proper digit is selected, the simulator determines whether read, write, or execute is available by subtracting the numeric values 4, 2, and 1. The result is a Boolean decision for the requested permission type.

This approach is compact and matches the way students usually learn Unix permissions. A permission value such as 640 can be interpreted directly: the owner can read and write, the group can read, and others have no access. The simulator uses this same interpretation to approve or deny operations. This also makes it easier to demonstrate permission changes during a presentation because the effect of `chmod` can be

observed immediately through `read`, `write`, `run`, `ls`, or directory navigation attempts.

The implementation also includes basic permission-value validation. Negative values and values greater than 777 are rejected before being stored as file or directory permissions. This improves robustness by preventing the most obvious invalid permission states, while a production-quality version would add stricter digit-by-digit validation and more complete error reporting for every malformed permission input.

VI. FILE AND DIRECTORY ENFORCEMENT

The simulator enforces file permissions through the `read`, `write`, and `run` commands. The `read` command locates the named file in the current directory and checks read permission before displaying file content. The `write` command checks write permission before replacing the file content. The `run` command checks execute permission before printing that the file executed successfully. In each case, failure results in an error message rather than the operation being performed.

Directory permissions are enforced differently because directories control structure rather than ordinary file content. The `mkdir` and `mkfil` commands require write permission on the current directory because creating a new entry changes the directory. The `rmdir` and `rmfil` commands also require write permission because deleting an entry alters the directory. The `ls` command requires read permission because it displays directory contents. The `opendir` command requires execute permission because execute on a directory represents traverse access.

This distinction between file permissions and directory permissions is one of the strongest parts of the simulator. Many students initially assume that read, write, and execute always mean the same thing. The project shows that directory permissions have different operational meanings: list, alter, and traverse. This is also why a user may be unable to reach a file even when that file itself appears to grant access. Without execute permission on the containing directory path, the file cannot be reached through normal traversal.

The directory class maintains separate lists of subdirectories and files. It checks for duplicate names before adding new entries. It also recursively cleans up files and subdirectories when a directory is deleted. This keeps the simulated filesystem tree consistent and prevents simple duplicate-entry errors.

VII. OWNERSHIP AND SECURITY MODIFICATION

The simulator includes `chmod`, `chown`, and `chgrp` because permission systems must control not only ordinary file actions but also changes to access-control metadata. The `chmod` command changes the numeric permission value of a file or directory. The `chown` command changes the owning user, and `chgrp` changes the owning group. In the simulator, these commands require the current user to be the owner of the object being modified.

This design is important because permission settings are security-sensitive. If any user could run `chmod`, then access control would be meaningless: a user denied access could simply grant themselves permission. By limiting modification commands to the owner, the simulator demonstrates the relationship between ownership and administrative control in a DAC-style model.

The identity commands also support security testing. The `login` command changes the active user, `joingrp` changes the

active group, and `whoami` displays the current user and group. These commands let the user test the same file or directory from different identities. For example, a file may be readable by its owner, unreadable to a user outside the group, and writable only after a permission change. This makes the simulator useful for showing how permission categories affect access.

VIII. TESTING AND VERIFICATION

The project was tested by using the command-line interface to create users, create groups, create directories, create files, change identities, and perform file and directory operations under different permission settings. The goal of testing was not only to see whether commands existed, but also to see whether the simulator enforced the correct permission rule for each operation.

Basic identity tests involved creating users with `adduser`, creating groups with `addgroup`, switching users with `login`, switching groups with `joingroup`, and checking the active identity with `whoami`. These tests verified that the simulator had a current user and group before permission checks were performed. User and group deletion commands were also tested to confirm that missing users or groups were rejected instead of silently ignored.

Filesystem tests involved creating directories with `mkdir`, creating files with `mkfil`, displaying the current path with `pwd`, listing contents with `ls`, opening directories with `opendir`, and returning with `backdir`. These tests verified that the simulated directory tree could be created, navigated, and modified. The tests also helped confirm that the root directory was created during startup and that subsequent directories were linked to parent directories.

Permission tests focused on reading, writing, executing, listing, altering, and traversing. The `read`, `write`, and `run` commands tested file permissions. The `ls` command tested directory read behavior. The `mkdir`, `mkfil`, `rmdir`, and `rmfil` commands tested directory write behavior. The `opendir` command tested directory execute behavior. The `chmod` command was used to create both allowed and denied cases.

The simulator also includes error-handling paths for invalid commands, incorrect argument counts, duplicate names, missing files, missing directories, invalid UID and GID values, and invalid permission values. During verification, the main demonstration path was run end-to-end to confirm that the program could build, launch, create an initial user and group, create directories and files, list contents, read and write file content, modify permissions, execute a file, navigate directories, and exit cleanly. The remaining error paths are important final verification cases because they show whether the program fails safely instead of performing an invalid operation.

IX. CORRECTNESS AND ROBUSTNESS ANALYSIS

For the simulator to be useful, it must do more than store files and directories. It must consistently apply the same security rule whenever an operation touches a protected object. The project addresses this by routing file and directory actions through permission-aware methods. File reading, writing, and running all call the file object's permission checks. Directory creation, file creation, deletion, listing, and traversal similarly depend on directory permissions. This matters because inconsistent enforcement is one of the most common weaknesses in access-control systems.

The implementation also separates ordinary object use from security modification. A user may be able to read or run a file without being allowed to change its permissions or owner. The `chmod`, `chown`, and `chgrp` functions require the active user to be the owner of the object before changing security metadata. This reflects an important distinction: access to an object is different from authority to redefine access for other users.

The simulator's input validation contributes to robustness. Commands check argument counts before using command parameters, and UID, GID, and permission values are parsed as integers before being accepted. The permission-setting logic rejects negative values and values outside the supported ugo range. These checks prevent the program from accepting obviously invalid security states.

A remaining robustness concern is that the simulator is still a student project rather than a hardened shell. Some commands assume that the active user, active group, and current directory already exist. This is reasonable because the startup routine creates them before the main command loop begins. In a production tool, however, additional defensive checks would be added before dereferencing pointers or traversing directory lists.

X. LIMITATIONS OF THE UGO MODEL AND ACLS

The ugo model is useful because it is small, predictable, and easy to display. It fits naturally into a three-digit permission value and is easy to teach. However, the model has a major limitation: it only provides one owner, one owning group, and one other-user category. This works for simple cases, but it becomes restrictive when a file needs different permissions for several specific users or several different groups.

For example, suppose a project file should be readable by one teaching assistant, writable by one team member, executable by a testing account, and hidden from everyone else. The ordinary ugo model cannot directly express that policy unless the administrator creates special groups or changes ownership. This can become awkward and difficult to maintain as the number of special cases increases.

Access Control Lists provide more fine-grained control. An ACL can attach multiple access entries to a file or directory, such as permissions for specific users and specific groups beyond the single owner and single owning group. This allows more precise policies without forcing every exception into the three ugo categories. POSIX ACLs extend the traditional model by adding named user and group entries while still preserving compatibility with basic permission bits [3].

The simulator does not implement ACLs, but the project helps explain why they exist. Once students understand how ugo works, it becomes easier to see why real systems sometimes need a more expressive model. The project therefore provides a foundation for understanding both traditional Unix permissions and more advanced access-control mechanisms.

XI. TEAM CONTRIBUTIONS AND PROJECT ORGANIZATION

The project was completed as a group effort. The implementation was divided across the main areas needed for a Unix-like permission simulator: user and group management, file and directory structures, command-line integration, documentation, testing, and final report

preparation. This division allowed different parts of the program to be developed independently and then connected through the main command interpreter.

The user and group code provides the identity foundation for the simulator. Without users, groups, UIDs, and GIDs, the permission system would not have identities to compare against file and directory metadata. The file and directory classes provide the simulated file system. They store objects, enforce permission checks, and support navigation and file operations. The main function connects these classes into an interactive console application.

The documentation and testing materials support the technical implementation by explaining command behavior, summarizing the permission model, and outlining expected results. These materials are useful for presentation, grading, and future maintenance because they explain not only what the simulator does but also why the design decisions matter.

XII. REQUIREMENT COVERAGE

The completed simulator addresses the main project requirements for a Unix-like file permission simulator. It creates users and groups, assigns each user a UID, assigns each group a GID, and uses those identities when evaluating access to simulated files and directories. It also creates a root directory and stores files and subdirectories below that root, which satisfies the requirement for a simple directory hierarchy.

The project implements the ugo permission model by storing a three-digit permission value with each file and directory. The first digit represents the owner, the second digit represents the group, and the third digit represents others. The simulator interprets each digit using the standard read, write, and execute values. This directly supports the required user, group, and others permission model.

The final command set also covers the expected file and directory operations. The `mkdir` command creates directories, `mkfil` creates files, `ls` lists simulated directory contents, and `chmod` changes permission values. The program also includes `rmdir`, `rmfil`, `pwd`, `opendir`, `backdir`, `read`, `write`, `run`, `chown`, and `chgrp`. These additional commands make the simulator more complete because they allow users to build a hierarchy, move through it, modify it, and test permission effects from different user and group identities.

The implementation also addresses the requirement that directory permissions behave differently from file permissions. Directory read permission is used for listing contents, directory write permission is used for adding or removing entries, and directory execute permission is used for traversal. This is a key security concept because access to a file depends not only on the file's own permissions but also on whether the user can move through the directory path that contains it.

XIII. DEMONSTRATION WORKFLOW

A clear project demonstration should begin with the initial login prompt. The presenter can create the first user, allow the program to create the default group, and then show that the simulator starts in the root directory. This establishes that every later command is evaluated from the perspective of a current user, current group, and current directory.

Next, the presenter can create a second user and group. After creating those identities, the presenter can use `mkdir` and `mkfil` to create a small directory tree and file. Running `ls` should show the files and directories in the current location,

along with permission and ownership information. This makes the simulated filesystem visible and connects the command output to the internal object model.

The strongest part of the demonstration is the permission contrast. A file or directory can be created under one user, then the active identity can be changed with `login` or `joingroup`. The presenter can then attempt read, write, run, `ls`, or `opendir` operations. Some commands should succeed and others should fail depending on the permission value. After using `chmod` to change the permission value, the same command can be repeated to show how access decisions change.

A directory-specific example is especially useful. The presenter can create a directory, remove execute permission for others, switch to another user, and then attempt to open that directory. Even if a file inside the directory would otherwise allow access, the user should be blocked from traversing the directory path. This directly demonstrates the difference between file permissions and directory permissions.

Finally, the demonstration can show `chown` and `chgrp`. Changing the owner or group of an object changes which permission digit applies during future checks. This helps explain why ownership is part of access control and why permission bits alone are not enough to understand the full access decision.

XIV. PRACTICAL SECURITY SCENARIOS

One realistic scenario for the simulator is a shared class project directory. A project owner may want teammates in the same group to read and execute files while preventing unrelated users from accessing or modifying the project. A permission value such as 750 demonstrates this case: the owner has read, write, and execute permission; the group has read and execute permission; and others have no permissions. If teammates also need to modify files, a value such as 770 would allow both the owner and group to read, write, and execute while still denying all access to others. The simulator makes this visible by letting the active user and group change before repeating the same operation.

Another scenario involves a private file. If a user creates a file and sets its permission value to 600, the owner should be able to read and write the file, but users outside the owner category should be denied. This shows how confidentiality and integrity can both be supported by permission settings. Read permission protects confidentiality, while write permission protects integrity.

A third scenario involves executable files. A file may be readable but not executable, or executable but not writable. This distinction matters in operating system security because running a program is different from viewing its contents or modifying it. The simulator's `run` command provides a simple way to show that execute permission is checked separately from read and write permission.

The directory traversal scenario is especially important. A file may appear to grant access, but if the user cannot traverse the containing directory, the file remains unreachable. This demonstrates why directory execute permission is not optional in a Unix-like model. It also shows that access control decisions are not always local to a single file; the directory path can affect whether the file can be reached at all.

These scenarios are useful because they connect abstract permission values to security outcomes. Instead of only memorizing that 7 means read, write, and execute, the user can observe how the permission value changes actual

command behavior. This makes the simulator a practical teaching tool rather than only a code exercise.

XV. COMPARISON WITH REAL UNIX SYSTEMS

The simulator is intentionally smaller than a real Unix filesystem, but the comparison to real Unix behavior is useful for analysis. In a real system, every process runs with user and group identity information, and filesystem operations are checked against inode metadata. The simulator mirrors this idea at a high level by storing a current user, a current group, and permission metadata for each simulated file or directory.

The `chmod` command in the simulator is similar in purpose to the real `chmod` utility because it changes the permission value associated with an object. However, the simulator only needs to update an in-memory object, while a real operating system updates persistent filesystem metadata. This difference is important because the simulator demonstrates the security concept without needing kernel-level privileges or real disk changes.

The `chown` and `chgrp` commands also resemble real Unix tools, but the simulator simplifies the authority model. In many real Unix-like systems, changing file ownership is more restricted than ordinary permission changes, often requiring elevated privileges. The simulator instead focuses on the educational rule that the owner controls access metadata. This keeps the project understandable while still showing why ownership matters.

The `ls` command in the simulator is similar to `ls -l` because it displays ownership and permission information. It does not reproduce every formatting detail of the real command, but it shows the information needed for this project: permissions, owner, group, and item name. This is enough to connect command output to permission decisions.

The simulator also differs from a real shell because it does not implement full path parsing. A real Unix shell can address complex relative and absolute paths, but this simulator mostly works from the current directory and uses `opendir` and `backdir` to move through the tree. This design is reasonable for a course project because it emphasizes permission behavior rather than full shell implementation.

Overall, the simulator is faithful to the core security model while simplifying implementation details that are outside the project's main objective. These simplifications make the system easier to present and test while preserving the important access-control concepts.

XVI. FUTURE IMPROVEMENTS

The simulator successfully demonstrates the major concepts of Unix-like file permissions, but several improvements could make it more realistic. One improvement would be to model users as members of multiple groups instead of tracking only one active group at a time. Real Unix-like systems allow a user to belong to several groups, and permission decisions may depend on all of those memberships.

Another improvement would be persistent storage. Currently, simulated users, groups, files, and directories exist only while the program is running. Saving the simulated state to a file and loading it again later would make the simulator more useful for longer demonstrations and repeated test cases.

A third improvement would be closer alignment with real Unix command names and behaviors. Commands such as `touch`, `rm`, `cd`, and `ls -l` could be added as aliases or

replacements for the simplified command names. This would make the simulator feel more familiar to users who already know Linux command-line tools.

Finally, ACL support could be added as an advanced extension. After the simulator demonstrates `ugo` permissions, ACLs would allow the project to show how a system can grant access to specific extra users or groups. This would directly address the limitations of the traditional model and deepen the cybersecurity analysis.

XVII. CONCLUSION

The File Permission Simulator successfully demonstrates the major ideas behind Unix-like file permissions. It includes users, groups, ownership metadata, permission values, a simulated directory tree, file operations, directory navigation, and permission modification commands. The project shows how read, write, and execute permissions affect both files and directories, and it demonstrates why directory execute permission is necessary for traversal.

From a cybersecurity perspective, the project demonstrates access control, least privilege, ownership, and separation between owner, group, and other users. It also shows why permission modification commands must be protected. If users could freely change ownership or permissions, the entire access-control model would lose its meaning.

The project is not a complete operating system, but it is an effective educational simulation. It provides a safe way to experiment with permission values and observe how access decisions change. It also creates a clear bridge between classroom concepts and real operating system security. By comparing the simulator's `ugo` model with Access Control Lists, the report shows that traditional Unix permissions are simple and useful, but less flexible when access rules must apply to specific users or groups.

REFERENCES

- [1] TimmyMcPickles. File Permission Simulator source code and README. GitHub repository, 2026. URL: https://github.com/TimmyMcPickles/file_perm_sim.
- [2] IEEE and The Open Group. chmod - change file modes. The Open Group Base Specifications Issue 7, 2018. URL: <https://pubs.opengroup.org/onlinepubs/9699919799/utilities/chmod.html>.
- [3] Andreas Gruenbacher. POSIX Access Control Lists on Linux. USENIX Annual Technical Conference, FREENIX Track, 2003.
- [4] Michael Kerrisk. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press, 2010.
- [5] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. Operating System Concepts. Wiley, 2018.
- [6] W. Richard Stevens and Stephen A. Rago. Advanced Programming in the UNIX Environment. Addison-Wesley, 2013.